Índices

Qué es un índice

Definición	Un índice, es una estructura que almacena datos copiados del campo de la tabla sobre el cual se definió el índice.

Su función, al igual que el índice de un libro, es permitir encontrar algo de forma más eficiente, en un libro será un capítulo del libro mientras que en tabla será un registro de la misma.

La forma de almacenar los índices varía de DBMS a DBMS, lo importante es que todos establecen la separación en el nivel físico.

La sintaxis básica para crear un índice es:

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ name ] ON table [ USING method ]
( { column | ( expression ) }[ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ...] )
[WHERE predicate]
```

- **Concurrently**: permite crear el índice sin bloquear la tabla
- **Method**: uno de btree, hash, gin, gist, spgist, brin. (Si no se especifica asume btree)
- **Expression**: permite definir una expresión para crear el índice.
- **Predicate**: expresión para definir un índice parcial.

Cómo funciona un índice

Cuando realizamos una consulta a la base de datos, el *Procesador de Consultas* (el encargado de generar los planes de consultas) define si debe recorrer la tabla completa (escaneo secuencial) o si utilizará alguno de los índices definidos para obtener los registros solicitados de forma más rápida.

No siempre el acceso a través de los índices es el más rápido, por ejemplo si la tabla posee pocos registros siempre será más rápido un acceso secuencial que un acceso a través de alguno de los índices definidos.

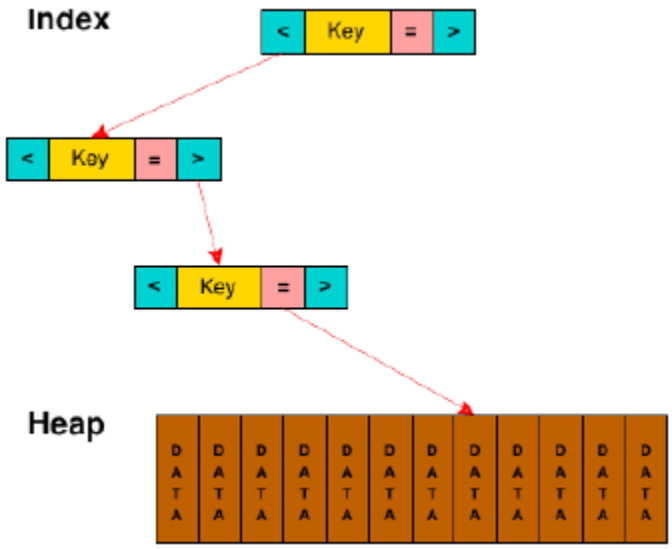
Otra cuestión a tener en cuenta es que cada actualización a una tabla (crear, modificar o eliminar un registro) impactará en los índices que la misma posea, por ende una tabla con muchos índices se traducirá en una baja en el rendimiento en las operaciones de alta, baja o modificación de registros.

Tipos de índices

Existen distintos tipos de índices, a continuación mencionamos algunos de los más comunes:

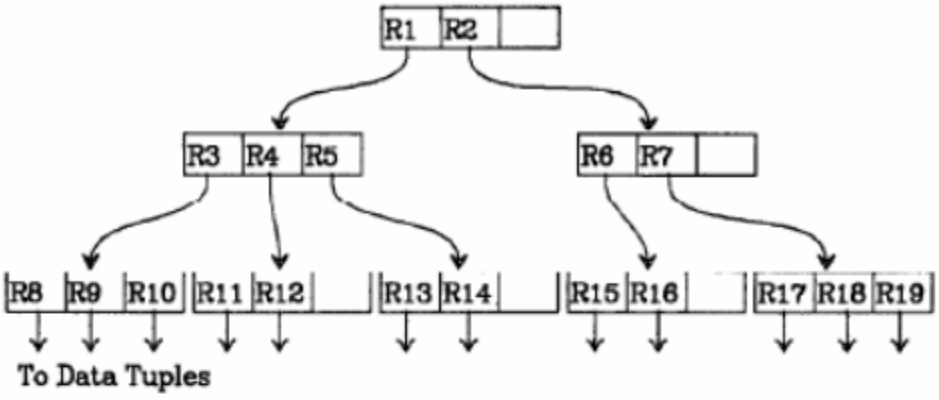
B-Tree

- Ideal sobre columnas que posean valores únicos.
- Puede proveer los datos ordenados
- Mejor performance en temas de concurrencia

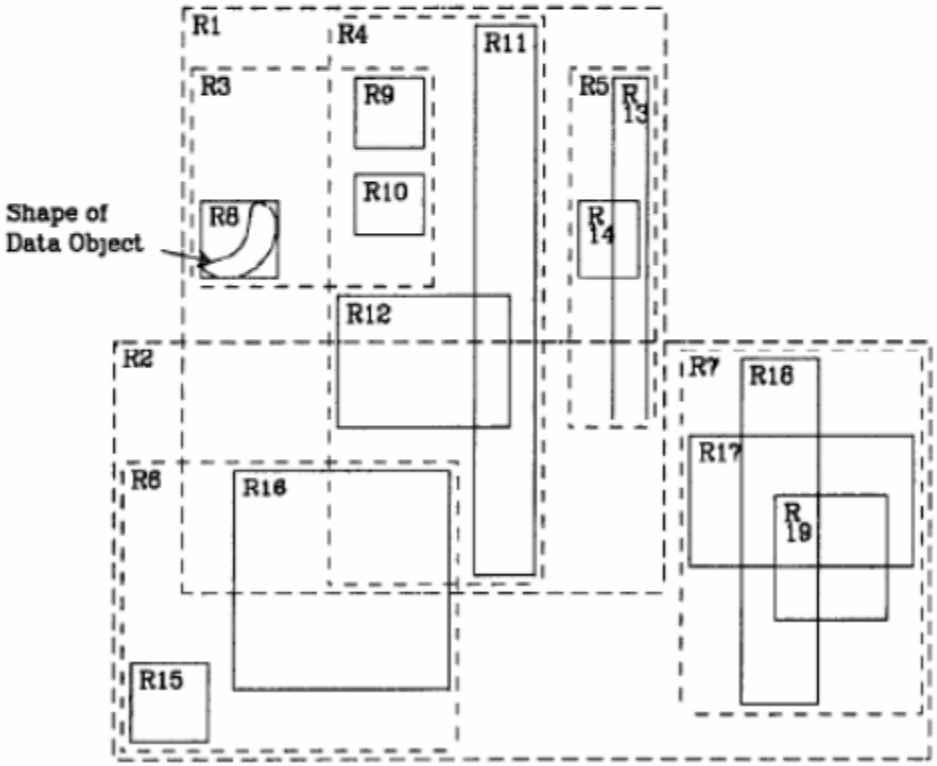


Generalized Search Tree (GIST)

- Ideal para indexar datos complejos que no se “adaptan” a un índice b-tree
- Permite indexar datos geográficos, arrays, par clave/valor



(a)



Generalized Inverted Index (GIN)

- Ideal para indexar valores compuestos
- Ideal para indexar columnas con muchos duplicados, ya que por cada clave puede tener punteros a varias filas. Esto también reduce el tamaño del índice
- Este índice guarda un conjunto de pares (clave, valor) donde el “valor” es un conjunto de IDs de filas donde ocurre esa clave
- Muy utilizado en valores de tipo: documento, JSON, arrays

Otros

- Índices por expresiones (Expression Index)
    - Permite definir el índice a partir de una expresión, función, concatenación, etc.
    - Ej: lower(nombre)
- Índices parciales (Partial Index)
    - Permite definir un índice para un subconjunto de filas y no toda la tabla
    - Reduce el overhead en operaciones de insert/update
    - Reduce el tamaño del índice
    - Ej: CREATE INDEX state_idx ON customer (state) WHERE state = 'AZ';

Resumen

- Sólo B-tree, GIN y GIST soportan índices multicolumna
- B-tree ideal sobre columnas que posean valores únicos.
- GIN Ideal para indexar columnas con muchos duplicados
- GIST para todo lo demás

Clustering

Toda las estructuras que maneja una base de datos son almacenadas finalmente en disco, por lo tanto, mientras más rápido sea el acceso a disco, más rápido va a poder devolver los datos que se le piden. Una forma de posibilitar esto es guardar los datos de forma contínua o lo más "cercanos" entre sí que se pueda. A esto se lo llama clustering y las base de datos lo realizan a partir de un índice que debe ser creado previamente.

Cuando se están manejando datos espaciales, esto es realmente útil debido a lo costoso de procesar dichos datos.

Sintaxis

```
CLUSTER [VERBOSE] table_name [ USING index_name ]
```

Ejemplo

```
CREATE INDEX geom_idx ON TABLE my_tabla_geo USING GIST(the_geom);

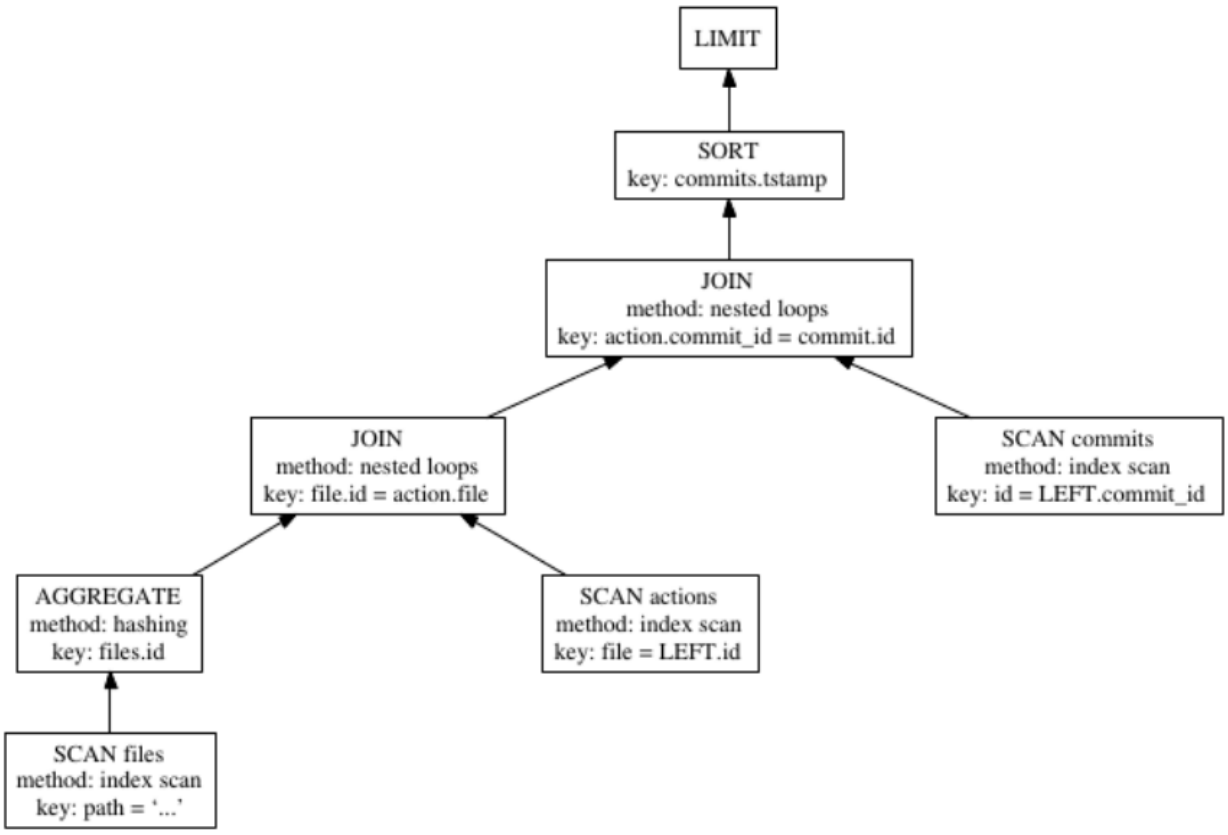
CLUSTER my_tabla_geo USING geom_idx;
```

Optimización de consultas

SQL es un lenguaje declarativo, esto significa que podemos decirle QUÉ hacer, pero no CÓMO. Por lo tanto, el motor de BD por cada consulta evalúa distintos planes de ejecución seleccionando el menos costoso.

El comando EXPLAIN nos permite ver, evaluar y mejorar dichos planes.

Ejemplo



¿Cómo ejecutarlo?

```
EXPLAIN select * from table_dummy;

EXPLAIN VERBOSE select * from table_dummy;
```

¿Cómo interpretar la salida?

```
QUERY PLAN
-------------------------------------------------------------
Seq Scan on table_dummy (cost=0.00..458.00 rows=10000 width=244)
```

Para cada consulta, el motor de base de datos arma un árbol de ejecución, donde cada nodo es un plan a ejecutar.

Por cada plan de ejecución que el motor realice se mostrará una fila (un nodo del árbol).

Para interpretar la salida del comando EXPLAIN, estos nodos deben leerse desde abajo hacia arriba.

Cada nodo presenta la siguiente información:

- Costo estimado para comenzar a devolver información
- Costo estimado para finalizar la consulta
- Cantidad estimadas de filas procesadas
- Cantidad estimados de bytes

¿Qué tiene que evaluar el planner?

- Scan Methods (sequential, index, bitmap index)
- Join Methods (nested loops, hash join, merge join)
- Aggregation Methods (hash, sort)

Scan Methods

Son los métodos que utiliza el motor de base de datos para leer (obtener y devolver) los datos de las tablas. Puede ser secuencial o indexado.

¿Importa la distribución de los datos?

Sí. Al motor “le importa” porque en base a eso, también determina qué método de escaneo realizar sobre la tabla.

No siempre un acceso indexado es más eficiente que un acceso secuencial. Por ejemplo, si el resultado de la consulta involucra más del 70% de los datos de la misma, entonces un acceso secuencial es más eficiente que uno indexado.

¿Puedo forzar el uso de índices?

Sí. Con los siguientes comandos:

- SET enable_seqscan = false;
- SET enable_bitmapscan = false;
- SET enable_nestloop = false

Ante acciones como estas, hay que prestar atención al impacto sobre las consultas.

Corolario: en caso de que el motor siga eligiendo un plan que no involucra el uso de índices, puede deberse a causas más fundamentales, como por ejemplo que la condición del query no utilice campos del índice.

Join Methods

Son los métodos que utiliza el motor de base de datos para unir/cruzar los datos de las tablas. Es una de las cosas más importantes a tener en cuenta en la optimización de consultas.

Nested loops

Puede ser tanto *sequential scan* como *index scan*

Sequential Scan

Ocurre ante un join entre 2 tablas y lo que hace el motor es comparar todas las filas de la tabla 1 contra todas las filas de la tabla 2. Sólo es útil cuando las tablas involucradas tienen pocas filas ya que lo realiza de forma muy rápida, tratar de evitar en otros casos.

En pseudocódigo sería algo como:

```
for (i = 0; i < length(t1); i++)
  for (j = 0; j < length(t2); j++)
    if (t1[i] == t2[j])
      output(t1[i], t2[j]);
```

Index Scan

¿Qué ocurre cuando creamos índices sobre las tablas y realizamos algún join estableciendo un filtro? En este caso, el motor va a realizar un nested loop, pero utilizando la información proporcionada por los índices. Con lo cual sólo va recorrer las filas que verifiquen la condición establecida en ambas tablas.

```
for (i = 0; i < length(outer); i++)
  index_entry = get_first_match(outer[j])
while (index_entry)
  output(outer[i], inner[index_entry]);
  index_entry = get_next_match(index_entry);
```

Corolario	no todos los nested loops son malos. Si se tiene un índice sobre la tabla no es necesariamente malo

Hash Join

Lo que hace es crear una estructura en memoria donde guarda el hash de las claves de 1 tabla y luego recorre dicha estructura efectuando comparaciones contra los valores de la otra tabla involucrada. Es uno de los joins más comunes elegidos por el motor. Es bastante efectivo siempre y cuando la tabla sobre la cual se genera el hash pueda ser almacenada en memoria.

Por lo general ocurre cuando se establece algún tipo de restricción sobre alguna tabla. Restricciones del estilo > o < pero no =.

En pseudocódigo sería algo como:

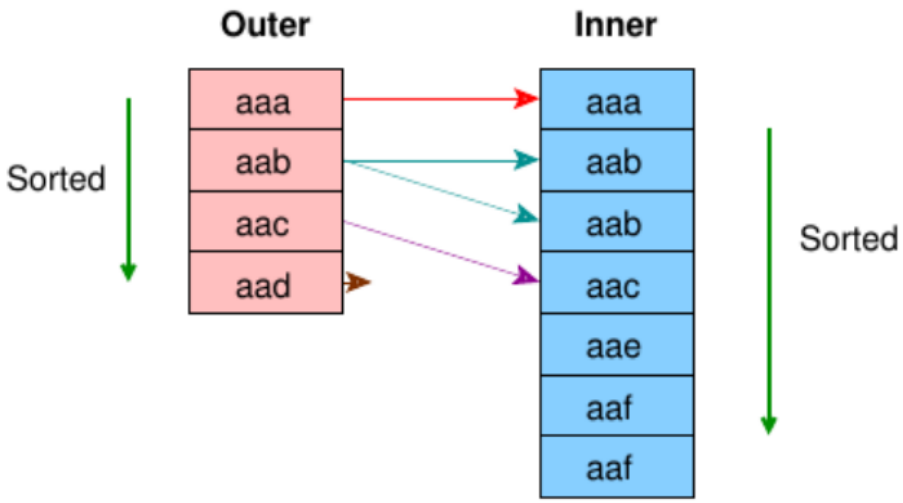
```
for (j = 0; j < length(inner); j++)
  hash_key = hash(inner[j]);
  append(hash_store[hash_key], inner[j]);

for (i = 0; i < length(outer); i++)
  hash_key = hash(outer[i]);
  for (j = 0; j < length(hash_store[hash_key]); j++)
    if (outer[i] == hash_store[hash_key][j])
      output(outer[i], inner[j]);
```

Merge Join

Lo que hace el motor en este caso es ordenar ambas tablas y luego las recorre casi en paralelo devolviendo las filas que machean la condición del join. Este tipo de join ocurre cuando se establece un join sin una restricción. Es bastante útil para grandes tablas ya que no requiere que

la tabla sea levantada en memoria y porque no se realizan múltiples recorridas sobre las tablas. Tener en cuenta que si las tablas poseen algún tipo de índice que establece un orden, entonces el motor evita ordenar las tablas previamente.



Comandos que ayudan al motor a mejorar los planes de consultas:

- ANALYZE: ayuda a recolectar estadísticas sobre la distribución de los datos
- VACUUM: limpia índices y tablas.

¿Cuál es la importancia de estos comandos?

Ayudan al motor a tener estadísticas sobre las tablas y sus datos y por ende a mejorar el plan de acción de cada consulta.

Sitio recomendado

- https://explain.depesz.com/

Bibliografía

- PostgreSQL Current Documentation. Disponible en: https://www.postgresql.org/docs
- Explaining the Postgres Query Optimizer. Disponible en http://momjian.us/main/writings/pgsql/optimizer.pdf
- Explaining the Postgres Query Optimizer Disponible en https://www.youtube.com/watch?v=P5iZri9s0WQ